

Persistent World Layer Specification

Appearance, Operative State, Provenance, and Committed History

Flyxion
Independent Researcher

September 2026

Version 0.2 (adds §7, an optional resonant-fold extension; all other sections unchanged from Version 0.1)

Abstract

This document converts the layered account of persistent worlds into an implementation-facing specification. It defines six distinct roles: appearance, operative state, committed history, observation, rendering, and semantic folding, while retaining provenance as a separate certified fold. It specifies authority boundaries, event and checkpoint envelopes, gate outcomes, deterministic fold requirements, replay claims, concurrency rules, failure handling, conformance tests, and language mappings for Rust and Go. The central constraint is that visual recurrence, operational continuity, and historical identity are different properties. Rendering is disposable and non-authoritative; operative state drives provisional evolution; committed history warrants identity at commitment boundaries; and current operative and provenance states are certified folds over the same causal event history. This is a normative engineering contract, not a claim that every interactive system requires the strongest persistence grade.

Contents

1	Status, Scope, and Normative Language	2
2	System Model	2
2.1	Authority matrix	3
3	Layer Contracts	3
3.1	Appearance	3
3.2	Operative state	4
3.3	Provenance and permission state	4
3.4	Committed history	4
4	Observation, Rendering, and Attestation	4
5	Event Model	5
5.1	Event envelope	5
5.2	Candidate lifecycle	6
6	Semantic Folds	6

7	Optional Extension: Resonant or Mnemonic Fold	7
7.1	Definition and update rule	7
7.2	Fold family	7
7.3	Authority boundary	8
7.4	Relationship to the withdrawn replay-profile claim	8
7.5	Conformance	8
8	Commit Protocol	9
9	Checkpoints, Recovery, and Replay	9
10	Failure Cases and Required Responses	10
11	Security and Retention	11
12	Rust Mapping	11
13	Go Mapping	13
14	Conformance Test Suite	14
15	Conformance Levels	15
16	Revision Guidance for the Framework	15
17	Compact Reference	16

1 Status, Scope, and Normative Language

This specification applies to interactive or simulated systems that claim persistence across sessions, processes, repairs, software versions, or distributed authorities. It is deliberately independent of storage engine, network protocol, rendering technology, and simulation language. A conforming implementation can be local or distributed, symbolic or neural, discrete or continuous.

The words **MUST**, **MUST NOT**, **SHOULD**, and **MAY** are normative. “History” means committed history unless explicitly qualified. “State” is never used without identifying whether it means operative state, provenance state, or a provisional state.

The specification does not require every numerical integration step, frame, sensor sample, or rejected packet to be stored forever. It requires explicit policies for deciding which facts become identity-bearing commitments, how retained summaries are certified, and how omissions or compression remain auditable.

2 System Model

At a certified causal frontier ϕ , a world instance is represented by

$$W_\phi = (H_\phi, M_\phi, Q_\phi, C_\phi),$$

where H_ϕ is the committed event history visible at the frontier, M_ϕ is the certified operative fold, Q_ϕ is the certified provenance and permission fold, and C_ϕ is checkpoint metadata, if a checkpoint

exists. An observer receives an appearance

$$x = R(M_\phi, Q_\phi, \omega, \rho),$$

where ω specifies observer identity, viewpoint, channel, and authorization, and ρ specifies disposable presentation conditions such as resolution, level of detail, sampling seed, compression, or accessibility format.

Between commitment boundaries, the simulation may evolve through provisional states

$$\tilde{M}(t + \Delta) = F_\Delta(\tilde{M}(t); V, \Xi).$$

Here V identifies applicable rule, model, schema, and solver versions, while Ξ identifies controlled stochastic inputs. A provisional state is not historically authoritative merely because it was used to compute the next step.

2.1 Authority matrix

Object	Authoritative for	Not authoritative for
Appearance x	What an observer was presented under declared render conditions	Object identity, ownership, causal history, or an instantaneous global state
Provisional operative state $\tilde{M}$	Continuing computation inside an uncommitted interval	Certified persistence, provenance, or recovery after rollback
Certified operative state M_ϕ	Current dynamic queries and transitions at frontier ϕ	The warrant or lineage of its values
Provenance state Q_ϕ	Current ownership, permission, attribution, audit, and standing queries	Complete historical evidence unless the fold retains it explicitly
Committed history H_ϕ	Which commitments occurred, their causal relations, and which folds are warranted	Efficient per-frame or per-tick computation
Checkpoint C_ϕ	Accelerated reconstruction from a named frontier under declared versions and tolerances	Replacement of the event history it summarizes

3 Layer Contracts

3.1 Appearance

Appearance is an observer-relative projection returned through a presentation channel. It includes images, audio, maps, text descriptions, instrument traces, administrative views, or network view models. Two appearances may be equal while their source worlds differ in state, provenance, or history. Conversely, one certified world may yield many valid appearances.

An appearance record MAY carry a content hash, frontier identifier, observer-policy version, renderer version, and presentation parameters. Those fields make the output reproducible or attestable; they do not make the output authoritative by themselves.

Invariant 1 (Non-authority of appearance). *Equality of rendered outputs never entails equality of operative state, provenance state, or committed history.*

3.2 Operative state

Operative state contains the smallest representation on which required transition and query operations are defined while preserving all distinctions those operations need. Typical members include identifiers, positions, velocities, fields, inventories, object relations, health values, environmental resources, and dynamically active constraints.

An implementation **MUST** distinguish certified operative state from provisional operative state. A certified state names the history frontier and fold implementation that warrant it. A provisional state names its certified base frontier and the uncommitted computation that produced it. Rollback of provisional state **MUST NOT** alter committed history.

Invariant 2 (Frontier binding). *Every certified M is bound to exactly one declared causal frontier ϕ , schema version, and physical-fold version. It is invalid to compare or replace certified states while omitting those bindings.*

3.3 Provenance and permission state

The provenance state Q is a current, queryable fold of historically active relations that affect meaning or admissibility without necessarily changing immediate dynamics. Examples include ownership, authorship, repair lineage, access rights, encumbrances, sanctions, disputed claims, evidence standing, and audit counters.

The separation between M and Q is logical, not necessarily physical. They may share a database or process. Their update functions **MUST** remain independently specified. A cross-layer consequence, such as permission opening a physical door, **MUST** occur through a typed admitted event or an explicitly versioned deterministic rule, never through an unrecorded side effect.

3.4 Committed history

History is an append-only set of immutable event envelopes with a causal order. A single-writer implementation **MAY** use a sequence. A distributed implementation **SHOULD** use a directed acyclic graph or another representation that preserves causal parents and concurrency without inventing a false wall-clock total order.

Appending a corrective event is permitted. Replacing, deleting, or semantically reinterpreting an existing event in place is forbidden. If legal or operational policy requires physical deletion, history **MUST** retain an auditable tombstone or deletion certificate sufficient to identify the affected commitment, authority, reason, and frontier; conformance then applies to the surviving logical history.

4 Observation, Rendering, and Attestation

An observation is sampled information available to an observer or sensor. Rendering is the computation that produces a presentation from certified folds and observer conditions. Attestation is the distinct operation by which selected observed content becomes evidence that can constrain future admissibility.

$$\begin{aligned} \text{observe} &: (M, Q, \omega, \rho) \rightarrow x && [\text{no commit}], \\ \text{attest} &: (x, \text{witness}, \text{rule}, \phi) \rightarrow e && [\text{candidate commit}]. \end{aligned}$$

Rendering **MUST** be side-effect-free with respect to M , Q , and H . Telemetry, access logs, billing records, or signed captures generated near a render call **MUST** be modeled as separate operations. A cache miss, frame request, or additional viewer **MUST NOT** change world history unless an independently named policy declares that access itself is a world event.

An attestation **MUST** identify the observed content or its content address, the witness or signing authority, the observation frontier if known, the rule under which it becomes constraining, and its verification status. Unverified material **MAY** be committed as an allegation or pending attestation, but **MUST NOT** acquire verified standing merely by entering the log.

Invariant 3 (Look/attest separation). *Repeated observation with identical certified inputs may change disposable caches and metrics, but cannot change authoritative folds. Only an admitted attestation event may make observed content historically constraining.*

5 Event Model

5.1 Event envelope

Each committed event **MUST** include the following logical fields, though encoding and field names may differ:

Field	Contract
Event identifier	Stable, globally unique or content-addressed identifier
World identifier	Namespace preventing cross-world replay
Kind and schema version	Typed payload discriminator and decoder version
Causal parents	Zero or more identifiers defining the event's immediate causal dependencies
Proposal reference	Identifier of the candidate being decided or transformed, when applicable
Disposition	Proposed, admitted, refused, deferred, repaired, attested, checkpointed, replay-discrepant, or a domain extension
Actor and authority	Principal initiating the candidate and authority admitting the event
Frontier	Causal frontier against which the gate evaluated the candidate
Payload	Canonically encoded domain data
Witness references	Bounded references to evidence not adequately represented by current folds
Versions	Gate, fold, rule, model, solver, and schema versions needed for interpretation
Integrity data	Payload hash and, where required, signatures or authenticated metadata
Time metadata	Optional physical time plus required logical or causal ordering data

Physical timestamps MUST NOT be used as the sole causal-ordering mechanism. Canonical encoding MUST define field ordering, number formats, string normalization, and treatment of unknown fields wherever hashes or signatures are compared.

5.2 Candidate lifecycle

A candidate begins outside authoritative history or as a committed PROPOSED event, according to domain policy. A gate evaluates

$$A(c \mid M_\phi, Q_\phi, \phi, W) \in \{\text{ADMIT}, \text{REFUSE}, \text{REPAIR}, \text{DEFER}\}.$$

The gate MUST be versioned. Its decision MUST name the evaluated frontier. Its witness set W MUST be bounded and content-addressed. Normal gate execution SHOULD use certified folds; direct history lookup is reserved for evidence whose exact prior form matters.

ADMIT appends an admitted event and applies every relevant fold. REFUSE appends a refusal under the configured retention policy and normally leaves M unchanged; it may update Q only through an explicit rule. REPAIR appends an event that references both the defective or conflicting antecedent and the replacement disposition; it never overwrites either. DEFER appends a reason and blocking condition, changes neither M nor ordinary permission standing by default, and creates an obligation governed by an explicit liveness policy.

A deferred candidate MUST eventually be resolved, cancelled by an authorized event, or remain visibly pending. A timeout MUST NOT silently discard it. Resolution MUST cite the deferral event as a causal parent.

Invariant 4 (Disposition visibility). *Every gate invocation that the deployment policy defines as identity-bearing has exactly one durable disposition. Retries are correlated with the original candidate identifier and cannot masquerade as unrelated proposals.*

6 Semantic Folds

A semantic fold is a deterministic, versioned reducer that converts a causally valid event history, or a checkpoint plus event suffix, into a queryable projection. The required authoritative folds are

$$M_\phi = \text{fold}(f_{\text{phys}}^v, M_0, H_\phi), \quad Q_\phi = \text{fold}(f_{\text{prov}}^u, Q_0, H_\phi).$$

Accounting, narrative, search, analytics, observer-index, and retention-summary folds MAY be added. A derived fold does not become authoritative merely because it is cached.

Each fold definition MUST declare its accepted event schemas, initial state, output schema, version, error behavior, ordering assumptions, treatment of unknown events, and migration policy. It MUST NOT consult undeclared ambient state such as current wall time, process-local randomness, network responses, mutable configuration, or unspecified floating-point behavior. Required external inputs must be captured in the event, checkpoint, or versioned execution profile.

If two events are concurrent, an implementation may apply them in either order only after demonstrating commutation for every authoritative fold they touch:

$$f_k(f_k(s, e_i), e_j) = f_k(f_k(s, e_j), e_i) \quad \text{for all affected } k.$$

Commutation in M alone does not imply commutation in Q . Conflicting events MUST be serialized, refused, repaired, or deferred according to a deterministic policy.

Invariant 5 (Prefix determinism). *Two conforming replicas using the same initial state, fold versions, canonical event content, and causally equivalent ordering of certified-independent events produce equivalent certified folds under the declared equality relation.*

Invariant 6 (Monotone history, selective folds). *Appending an event strictly advances logical history, but need not change every fold. A refusal or deferral normally lies in the kernel of the physical fold while remaining visible to provenance or pending-work folds.*

7 Optional Extension: Resonant or Mnemonic Fold

This section is non-normative for base conformance (Section 15) and describes how a MEM|8-style continuously maintained associative memory can be added to the architecture without weakening any invariant already stated. Nothing in this section is required of a conforming implementation that does not adopt it.

The motivating question is whether a “constant resonating pool” – a bounded, continuously updated memory whose present configuration reflects the accumulated effect of prior activity through resonance, interference, decay, and reinforcement – is compatible with a history-primary architecture, given that rendering under Section 3.1 **MUST NOT** scan H_ϕ on every frame. It is compatible, provided the pool is understood as a continuously maintained present disposition rather than as the complete history being rescanned, and provided its authority is bounded exactly as M_ϕ ’s and Q_ϕ ’s are bounded elsewhere in this specification.

7.1 Definition and update rule

An implementation **MAY** define a resonant fold Ψ_ϕ , distinct from the replay-profile vector P of Section 9 (the two are unrelated objects that happen to share initials in informal discussion; this specification reserves P for the replay profile and Ψ for the resonant fold to avoid the collision). The resonant fold is updated incrementally by each admitted event:

$$\Psi_{t+1} = U(\Psi_t, E_{t+1}),$$

where U is a versioned, deterministic update rule combining the prior pool configuration with the newly admitted event’s effect through whatever combination of resonance, interference, decay, and reinforcement the domain defines. Rendering **MAY** then read the pool alongside the other certified folds:

$$x_t = R(M_t, Q_t, \Psi_t, \omega_t).$$

Ψ_t therefore determines salience, expectation, associative readiness, and which continuations are easiest to recruit during presentation. It extends Section 3.1’s renderer signature by one argument; it does not replace M_t or Q_t in that signature, and an implementation that does not adopt this extension continues to render from (M_t, Q_t, ω_t) exactly as specified there.

7.2 Fold family

Adopting this extension produces a three-fold family over the same committed history, rather than the two-fold family of Section 6:

$$H_\phi \longrightarrow \begin{cases} M_\phi & \text{physical fold,} \\ Q_\phi & \text{provenance fold,} \\ \Psi_\phi & \text{resonant or mnemonic fold,} \end{cases} \quad x_t = R(M_t, Q_t, \Psi_t, \omega_t).$$

Ψ_ϕ MUST be declared with the same fold obligations as any other fold under Section 6: a declared accepted-event schema, initial state, output schema, version, and treatment of unknown events. Unlike M and Q , Ψ MAY be lossy and non-invertible by design. A pool that remained constant in informational content while retaining arbitrary history exactly would not be a resonant fold in the intended sense; it would be a second copy of H wearing different notation. The pool MAY remain constant in capacity, dimensionality, and per-frame access cost only by being selective: some distinctions persist, some decay, and some become inseparably superposed. This lossiness is expected and unproblematic exactly because Ψ is never the layer of record.

7.3 Authority boundary

Invariant 7 (Resonant fold non-authority). Ψ_t SHOULD influence rendering and MAY influence heuristic prefiltering of candidates, but it MUST NOT be consulted by the admissibility gate A of Section 5 as a substitute for evidence drawn from Q_ϕ or from a bounded witness set W . A decision that would differ depending on the current configuration of Ψ_t but not on M_ϕ , Q_ϕ , or W MUST NOT be an admit, refuse, repair, or deferral disposition; it may only affect which candidates are surfaced for evaluation, never which are admitted.

This is the same boundary Invariant 1 (non-authority of appearance) already draws for x_t , extended to a second disposable-but-influential object. Ψ_t is closer to appearance than to state in this respect: it is downstream of H_ϕ and causally shaped by it, exactly as required for the pool to be a faithful fold rather than an arbitrary cache, but it is not itself a source of admissibility, ownership, or identity. If reconstruction, audit, or dispute requires the precise past, the system MUST consult H_ϕ through M_ϕ or Q_ϕ ; ordinary rendering or heuristic recall MAY proceed from Ψ_t alone. An implementation MUST NOT allow content that exists only in Ψ_t – and never became an admitted or attested event – to acquire evidentiary standing merely because it is dynamically available for recall. The look/attest separation of Invariant 2 applies to the resonant fold without modification: resonance is a form of looking, however elaborate, until an attestation commits it.

7.4 Relationship to the withdrawn replay-profile claim

Section 16 withdraws, as an implementation claim, the suggestion that replay profiles $P = (p_b, p_n, p_s, p_c, p_p)$ form a useful join-semilattice, on the grounds that a componentwise maximum can combine guarantees achieved by different artifacts or executions and thereby describe no actual replay. The resonant fold does not reintroduce that error under a different name. Ψ_t is not a summary of verification strength across runs; it is a single system’s own accumulated disposition, updated by one deterministic rule along one causal history. It is a fold in the same sense M_ϕ and Q_ϕ are folds (Section 6), not an aggregate statistic assembled after the fact across incomparable executions. The two constructions address different axes – one asks how strongly a replay is verified, the other asks what a bounded present carries forward from an unbounded past – and a conforming implementation MUST NOT conflate them by, for instance, treating a high-salience entry in Ψ_t as evidence of a high replay grade for the event that produced it.

7.5 Conformance

Adopting this extension does not raise or lower an implementation’s conformance level under Section 15; the levels defined there are orthogonal to it. An implementation MAY additionally claim **Resonant** compatibility by satisfying Invariant 7 and passing test **T13** below.

T13. Resonant non-authority. Populate Ψ with content derived only from observation, never from an attested event. Confirm that every admit, refuse, repair, and deferral disposition produced by the gate is unchanged from a run with Ψ held empty, and that no query against M_ϕ or Q_ϕ can retrieve that content as if it were certified.

8 Commit Protocol

The minimal commit protocol has six phases. The implementation captures a candidate and its idempotency key; reads a certified (M, Q, ϕ) ; evaluates the versioned gate; constructs and durably appends one disposition event; applies or invalidates relevant fold caches; and publishes the new certified frontier. A crash before durable append leaves no commitment. A crash after durable append but before cache update is recovered by replay. Publication **MUST NOT** expose a certified frontier whose required authoritative folds are unavailable or fail verification.

Append and fold-cache update need not share one database transaction, but the history append is the commitment point. Cache records **MUST** name their frontier and fold version. Consumers **MUST** reject a cache whose frontier is neither equal to nor an explicitly acceptable ancestor of the requested frontier.

Idempotency is mandatory at the candidate and event levels. Reprocessing the same candidate **MUST** return or reference its prior disposition rather than append a second logically independent admission. Content-identical candidates with distinct identifiers remain distinct proposals unless domain policy explicitly coalesces them.

9 Checkpoints, Recovery, and Replay

A checkpoint is an accelerator bound to

$$C = (\text{id}, \phi, M_\phi, Q_\phi, V, \Xi, T, h).$$

The tuple records its identifier, causal frontier, authoritative folds, software and schema versions, controlled stochastic inputs, comparison policies or tolerances, and integrity hashes. A checkpoint **MUST** be derivable from or certified against history. It **MUST NOT** replace, truncate, or contradict history without a separately committed and auditable retention or migration event.

Recovery loads a verified checkpoint, verifies all declared bindings, and applies a causally valid suffix. Failure to verify a checkpoint or replay result **MUST** append or externally preserve a `REPLAY_DISCREPANT` record before a repaired checkpoint is promoted. The earlier artifact remains addressable, even if quarantined.

Replay claims are a vector, not a scalar:

$$P = (p_b, p_n, p_s, p_c, p_p),$$

covering bitwise, numerical, semantic, causal, and provenance recurrence. Each component is a predicate with declared inputs, environment, equality relation, tolerance, and evidence. A system **MUST** report components independently. It **MUST NOT** infer semantic or provenance recurrence from bitwise or visual equality, nor claim a combined profile assembled from successes obtained by different runs unless it labels that profile as an aggregate rather than a single-run guarantee.

10 Failure Cases and Required Responses

Failure	Detection	Required response
Beautiful reset	Visual scene recurs but identifiers, repairs, or ownership do not	Reject persistence claim; restore from certified folds or record reconstruction as repair
Authoritative screenshot	Presentation is used as proof of complete or instantaneous global state	Require frontier and source-fold evidence; downgrade claim to presentation evidence
Invisible refusal	Rejected attempts leave no governed audit trace	Apply refusal retention/aggregation policy and update explicit provenance fold if authorized
Silent timeout	Deferred candidate disappears without disposition	Recover pending candidate and append cancellation, refusal, or resolution linked to the deferral
Repaired past	Divergent checkpoint is overwritten by a regenerated artifact	Preserve discrepancy and original identity; append repair and issue a new checkpoint
Poisoned ledger	Adversarial invalid proposals exhaust storage or reputation state	Apply versioned rate, aggregation, identity, and retention policies; certify summaries and their frontiers
Stale fold	Cache is served from a different or unknown frontier	Reject or label as stale; rebuild from a verified ancestor
Cross-fold race	Concurrent events commute in physics but conflict in ownership or permission	Treat as non-independent and serialize, refuse, repair, or defer
Schema drift	Old event gains new meaning under a new decoder	Retain old decoder semantics or commit an explicit migration producing a new fold version
Ambient nondeterminism	Replay changes because reducer consulted time, random state, locale, or network	Capture input, pin execution profile, append discrepancy, and recertify
Observer-caused growth	Each render or sensor read appends world events	Separate telemetry from attestation and restore pure-render contract
Unauthorized attestation	Unsigned or ineligible observation alters admissibility	Mark as allegation/pending, reverse only through compensating events, and preserve incident lineage
Resonant leakage	Content present only in an optional resonant fold Ψ (Section 7) is treated as certified evidence or changes a gate disposition	Reject the disposition, require an independent attestation, and audit the gate implementation for an undeclared Ψ dependency

11 Security and Retention

Append-only does not mean accept-all. Event admission MUST authenticate the submitting principal where identity matters, authorize the claimed operation, validate schema and size, enforce namespace separation, and resist replay through candidate identifiers or nonces. Signatures attest to a signer and byte sequence; they do not establish truth. Truth or standing is assigned only through a declared gate and provenance rule.

Refusal retention is policy-sensitive because complete storage can enable denial of service while complete omission destroys evidence. A conforming policy MUST state which refusals are retained individually, aggregated, sampled, or discarded; how aggregates bind to a frontier and rule version; and which security-relevant lineage cannot be compressed. Compression of history is itself a committed transformation with source range, algorithm version, output hash, custodian, and reversibility class.

Observer-specific rendering MUST enforce Q at the frontier from which M is rendered. It MUST NOT render from one frontier while authorizing from an incompatible frontier unless the result is explicitly labeled stale and the disclosure risk is acceptable by policy.

12 Rust Mapping

The following mapping is illustrative but preserves the normative boundaries. Domain payloads should use canonical serialization rather than relying on compiler layout.

```
pub struct Frontier(pub Vec<EventId>);
pub struct EventId(pub [u8; 32]);
pub struct CandidateId(pub [u8; 16]);

pub struct EventEnvelope<P> {
    pub id: EventId,
    pub world: WorldId,
    pub schema: SchemaId,
    pub parents: Vec<EventId>,
    pub proposal: Option<CandidateId>,
    pub disposition: Disposition,
    pub actor: PrincipalId,
    pub authority: PrincipalId,
    pub evaluated_at: Frontier,
    pub payload: P,
    pub witnesses: Vec<WitnessRef>,
    pub versions: Versions,
    pub integrity: Integrity,
    pub physical_time: Option<Timestamp>,
}

pub enum Disposition {
    Proposed, Admitted, Refused(RefusalCode),
    Deferred { reason: DeferralCode, until: Condition },
    Repaired { antecedents: Vec<EventId> },
    Attested(Verification), Checkpointed(CheckpointId),
}
```

```

    ReplayDiscrepant(DiscrepancyId),
}

pub struct Certified<S> {
    pub value: S,
    pub frontier: Frontier,
    pub fold_version: FoldVersion,
    pub digest: Digest,
}

pub struct Provisional<S> {
    pub value: S,
    pub base: Frontier,
    pub execution: ExecutionProfile,
}

pub trait Fold<E, S> {
    type Error;
    fn version(&self) -> FoldVersion;
    fn apply(&self, state: &mut S, event: &E) -> Result<(), Self::Error>;
}

pub trait Gate<C, M, Q, P> {
    type Error;
    fn version(&self) -> GateVersion;
    fn decide(
        &self, candidate: &C, m: &Certified<M>, q: &Certified<Q>,
        witnesses: &[WitnessRef]
    ) -> Result<Decision<P>, Self::Error>;
}

pub trait Renderer<M, Q, O, X> {
    type Error;
    fn render(
        &self, m: &Certified<M>, q: &Certified<Q>, observer: &O,
        presentation: &PresentationProfile
    ) -> Result<X, Self::Error>;
}

pub trait History<E> {
    type Error;
    fn append(&self, expected: &Frontier, event: E)
        -> Result<Frontier, Self::Error>;
    fn suffix(&self, after: &Frontier) -> Result<Vec<E>, Self::Error>;
}

```

Rust implementations SHOULD make certified and provisional states distinct types, keep renderer access immutable, use exhaustive matching for dispositions, and return frontier conflicts as

typed errors. The history implementation owns atomic append and idempotency. A fold does not append events; a gate does not mutate folds; orchestration is responsible for the commit protocol.

13 Go Mapping

Go cannot encode all boundaries in the type system, so interfaces and constructors must enforce them explicitly.

```
type Frontier struct { Heads []EventID }
type EventID [32]byte
type CandidateID [16]byte
```

```
type DispositionKind uint8
const (
    Proposed DispositionKind = iota
    Admitted
    Refused
    Deferred
    Repaired
    Attested
    Checkpointed
    ReplayDiscrepant
)
```

```
type EventEnvelope[P any] struct {
    ID          EventID
    World        WorldID
    Schema       SchemaID
    Parents      []EventID
    Proposal     *CandidateID
    Disposition  Disposition
    Actor        PrincipalID
    Authority    PrincipalID
    EvaluatedAt  Frontier
    Payload      P
    Witnesses    []WitnessRef
    Versions     Versions
    Integrity    Integrity
    PhysicalTime *time.Time
}
```

```
type Certified[S any] struct {
    Value        S
    Frontier     Frontier
    FoldVersion  FoldVersion
    Digest       Digest
}
```

```

type Provisional[S any] struct {
    Value      S
    Base       Frontier
    Execution  ExecutionProfile
}

type Fold[E, S any] interface {
    Version() FoldVersion
    Apply(context.Context, *S, E) error
}

type Gate[C, M, Q, P any] interface {
    Version() GateVersion
    Decide(context.Context, C, Certified[M], Certified[Q],
        []WitnessRef) (Decision[P], error)
}

type Renderer[M, Q, O, X any] interface {
    Render(context.Context, Certified[M], Certified[Q], O,
        PresentationProfile) (X, error)
}

type History[E any] interface {
    Append(context.Context, Frontier, E) (Frontier, error)
    Suffix(context.Context, Frontier) ([]E, error)
}

```

Constructors SHOULD defensively copy frontier heads and event slices. Implementations MUST NOT expose mutable maps or slices from certified values if callers could modify them without a commit. Disposition validation must reject impossible combinations, such as `Deferred` without a reason or `Repaired` without antecedents. As in Rust, the orchestrator composes history, gates, folds, and rendering; none silently performs another component's role.

14 Conformance Test Suite

An implementation claiming conformance MUST publish which tests apply and provide machine-verifiable evidence for each supported deployment mode.

- T1. Pure render.** Hash M , Q , and the history frontier before and after repeated rendering by multiple observers. All authoritative hashes and the frontier remain unchanged.
- T2. Attestation boundary.** Observe identical content twice, attest only once, and verify that exactly one attestation lineage can constrain the gate.
- T3. Fold reconstruction.** Rebuild M and Q from genesis and from each supported checkpoint. Results satisfy their declared equivalence relations at the same frontier.

- T4. Idempotent retry.** Submit the same candidate identifier repeatedly across simulated crashes. At most one logical disposition is admitted.
- T5. Refusal selectivity.** Refuse a candidate and confirm the physical fold is unchanged while the configured provenance or audit fold records exactly the policy-authorized effect.
- T6. Deferral liveness.** Defer under a missing witness or partition, restore the condition, and verify that resolution cites the original deferral. A timeout cannot erase the candidate.
- T7. Cross-fold concurrency.** Generate event pairs that commute in M but conflict in Q . The system does not certify them as independent.
- T8. Checkpoint corruption.** Alter a checkpoint byte. Recovery rejects it, preserves its identity, and records a discrepancy before promoting any repair.
- T9. Version drift.** Replay under a new schema, fold, solver, or model version. The system either reproduces the declared profile or reports which component failed.
- T10. Off-screen identity.** Remove an entity from all appearances, advance the world, and restore visibility. Identity and relevant properties derive from state and history, not regenerated resemblance.
- T11. Observer isolation.** Render authorized and unauthorized views from the same frontier. Hidden content does not leak, and neither render changes permissions.
- T12. History immutability.** Attempt in-place modification, deletion, reparenting, and reinterpretation. Each is rejected or represented as a new authorized event without changing prior event identity.

15 Conformance Levels

Layered conformance requires explicit appearance, certified/provisional operative state, and pure rendering. **Historied** conformance additionally requires immutable typed events, deterministic reconstruction, dispositions, and checkpoint binding. **Provenance-preserving** conformance additionally requires a certified Q fold and cross-fold conflict checks. **Distributed** conformance additionally requires causal frontiers, certified independence, partition-aware deferral or another explicit consistency policy, and replica convergence tests. An implementation **MUST NOT** claim a stronger level while omitting one of its obligations. An implementation **MAY** separately claim **Resonant** compatibility (Section 7) by satisfying Invariant 7 and passing test T13; this label is orthogonal to the four levels above and neither raises nor lowers any of them.

These conformance levels describe engineering guarantees, not a metaphysical ranking. A short-lived game or visualization may rationally stop at the layered level. A collaborative scientific, economic, legal, or archival world will usually require provenance-preserving or distributed conformance.

16 Revision Guidance for the Framework

The next conceptual revision should preserve three sharp separations. First, “observation” should name disposable sampling, while “attestation” names the identity-bearing act that makes selected content constraining. Second, “state” should be split everywhere into certified operative state,

provisional operative state, and provenance state; an unqualified s_t obscures authority. Third, “history-primary” should not imply that rendering scans the log or that solver microsteps are events. History warrants folds at commitment boundaries; certified folds serve current queries; provisional state drives computation between boundaries.

The event vocabulary should treat proposal status and disposition as related but distinct. In some deployments a proposal is itself historically significant; in others it remains transient until the gate decides. Both are permitted if the retention policy is explicit. Deferral should remain first-class because it represents epistemic insufficiency rather than failure, and repair should always preserve its antecedents.

Replay should remain multidimensional. The earlier suggestion that replay profiles form a useful join-semilattice should be withdrawn as an implementation claim: a componentwise maximum can combine guarantees achieved by different artifacts or executions and thereby describe no actual replay. The safe object is an evidence-bearing vector attached to one declared run, plus separately labeled aggregates if needed.

Finally, the framework should distinguish invariants from aspirations. Pure rendering, frontier binding, deterministic folds, causal-parent integrity, idempotent commitment, and visible repair are enforceable invariants. Long-term identity, semantic equivalence, and the adequacy of a chosen event vocabulary are domain-relative claims that require declared tests and evidence.

17 Compact Reference

At runtime, render only from compatible certified folds; treat the result as presentation, not state. Evolve provisional physics from a named certified base. At a commitment boundary, evaluate a versioned gate against compatible M , Q , and frontier values. Append exactly one visible disposition using an idempotency key. Rebuild or advance every affected fold deterministically. Publish the new frontier only when required folds are available. Checkpoint the folds with their frontier, versions, inputs, tolerances, and hashes. On disagreement, append discrepancy and repair; never rewrite the past to resemble the desired present.